```
The CRM team wants to understand the sequence in which customers
placed their orders.


For every order, display:


CustomerID --> customer
OrderID --> orders
OrderDate --> orders
The order number for that customer


For example:


Customer C001
First order  → 1
Second order → 2
Third order  → 3


The numbering should be based on OrderDate, from oldest to
newest.


select *
from `e1.orders`
limit 5
```

```
select *
from `e1.order_items`
limit 5



Find the total amount successfully paid by each customer.

Display:

CustomerID --> customers
Customer name --> customers
Number of successful payments --> payments
Total successful payment amount --> payments

customers --> orders --> payments

Show only customers whose successful payment amount is greater
than ₹50,000. --> having



select
  c.CustomerID,
  c.Name,
  count(p.PaymentID) as no_of_payments,
```

```
  sum(p.amount) as total_amt
from `e1.customers` as c
inner join `e1.orders` as o
on c.CustomerID = o.CustomerID
inner join `e1.payments` as p
on p.OrderID = o.OrderID
where p.Status = "Success"
group by
  c.CustomerID,
  c.Name
having total_amt > 50000
order by total_amt desc



For each category, calculate:

Category name --> category
Number of products --> products
Average MRP --> products
Average selling price --> products
Average discount amount --> mrp - sp --> products

Consider:

Discount = MRP - SellingPrice

Display categories with at least 5 products, --> having
```

```sql
select
  c.CategoryName,
  count(p.ProductID) as prd_cnt,
  round(avg(p.MRP),2) as avg_mrp,
  round(avg(p.SellingPrice),2) as avg_sp,
  round(avg(p.MRP - p.SellingPrice),2) as avg_discount
from `e1.products` as p
inner join `e1.categories` as c
on p.CategoryID = c.CategoryID
group by
  c.CategoryName
having prd_cnt >= 5
order by avg_discount desc
```

For every supplier, calculate:

Supplier name
Number of products supplied
Number of order-item records
Total quantity sold
Total sales value

Include suppliers even if they currently have no sales.

```
Sort by total sales from highest to lowest.




select s.SupplierName,
       count(distinct p.ProductID) as no_of_prds,
       count(oi.OrderItemID) as no_of_oi_rcrds,
       coalesce(sum(oi.Quantity),0) as total_qty_sold,
       coalesce(sum(oi.Total),0) as total_sales_value
from `e1.suppliers` as s
left join `e1.products` as p
on s.SupplierID = p.SupplierID
left join `e1.order_items` as oi
on p.ProductID = oi.ProductID
group by s.SupplierName
order by total_sales_value desc;




For every warehouse, calculate:

Warehouse name
Number of orders handled
Total inventory stock
Average inventory stock
```

Include warehouses even if they have no orders.

```sql
select w.Warehouse,
       count(distinct o.OrderID) as no_of_orders_handled,
       coalesce(sum(i.Stock),0) as total_inventory_stock,
       coalesce(round(avg(i.Stock),2),0) as avg_inventory_stock
from `e1.warehouses` as w
left join `e1.orders` as o
on w.WarehouseID = o.WarehouseID
left join `e1.inventory` as i
on w.WarehouseID = i.WarehouseID
group by w.Warehouse
```

Find the category whose products have the highest average selling price.

Display:

CategoryID
CategoryName
Average selling price

```sql
select
  c.CategoryID,
  c.CategoryName,
  round(avg(p.SellingPrice)) as avg_sp
from e1.categories as c
inner join e1.products as p
  on p.CategoryID = c.CategoryID
group by
  c.CategoryID,
  c.CategoryName
order by avg_sp desc
limit 1
```

For every supplier, rank their products according to selling price, from highest to lowest.

Display:

Supplier name
Product name
Selling price
Rank within supplier

```sql
SELECT
  s.supplierName,
  p.productName,
  p.SellingPrice,
rank() over(partition by s.supplierID order by p.SellingPrice
desc) as selling_rank
from `e1.products` as p
inner join `e1.suppliers` as s
on p.supplierID = s.supplierID
```

```
date --> 2024-01-15

yyyy-mm-dd

select current_date() as today
```

```
show the details of the customers who have joined in the recent
1000 days

select
  Name,
  SignupDate
```

```
from `e1.customers`
where SignupDate >= date_sub(current_date(), interval 1000 day)



select
  OrderID,
  OrderDate,
  extract(year from orderDate) as order_year,
  extract(month from orderDate) as order_month,
  extract(day from orderDate) as order_day
from `e1.orders`




How many orders were placed in each year?

select
  extract(year from orderDate) as order_year,
  count(*) as no_of_orders
from `e1.orders`
group by order_year
order by order_year desc







How many orders were placed in each month?
```

```sql
select
  extract(month from orderDate) as order_month,
  count(*) as no_of_orders
from `e1.orders`
group by order_month
order by order_month desc
```

```sql
select
  extract(year from orderDate) as order_year,
  extract(month from orderDate) as order_month,
  count(*) as no_of_orders
from `e1.orders`
group by order_year, order_month
order by order_year desc, order_month desc
```

```sql
date1 = 2024-03-15
date_trunc('2024-03-15', month)
```

```sql
select
  OrderDate,
  date_trunc(OrderDate,MONTH)
from `e1.orders`
```

```sql
select
  OrderDate,
  date_trunc(OrderDate,YEAR)
from `e1.orders`
```

What were the total sales for each month?

```sql
select
  date_trunc(OrderDate, MONTH) as sales_month,
  sum(oi.Total) as total_sales
from `e1.orders` as o
inner join `e1.order_items` as oi
  on o.OrderID = oi.OrderID
group by sales_month
order by sales_month
```

```sql
select DATE_DIFF(DATE '2024-01-20', DATE '2024-01-10', DAY) as
day_diff
```

Find customers who have placed at least 5 orders.

Display:

CustomerID
Customer name
City
Number of orders

For every customer, number their orders chronologically.

Display:

CustomerID
OrderID
OrderDate
Order number

For example:

Customer C001
Order O001 → 1

```
Order O005 → 2
Order O010 → 3
```

How many days has each customer been registered?

```
select
  CustomerID,
  SignupDate,
  DATE_DIFF(current_date(), SignupDate,DAY) as days
from `e1.customers`
```